VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Microprocessors - Microcontrollers

Report lab 3

Buttons/Switches

Advisor(s): Phan Van Sy
Student(s): Tran Le Dang Khoi 2452599

HO CHI MINH CITY, OCTOBER 2026

Contents

1 Exercise 1 **5**
1.1 Requirement . . . 5
1.2 FSM Design . . . 5

2 Exercise 2 **6**
2.1 Requirement . . . 6
2.2 Circuit Schematic . . . 6

3 Exercise 3 **7**
3.1 Requirement . . . 7
3.2 STM32 Configuration . . . 7
3.3 Source Code . . . 10

4 Exercise 4 **11**
4.1 Requirement . . . 11
4.2 Source Code . . . 11

5 Exercise 5 **12**
5.1 Requirement . . . 12
5.2 Source Code . . . 12

6 Exercise 6 **13**
6.1 Requirement . . . 13
6.2 Source Code . . . 13

7 Exercise 7 **14**
7.1 Requirement . . . 14
7.2 Source Code . . . 14

8 Exercise 8 **15**
8.1 Requirement . . . 15
8.2 Source Code . . . 15

9 Exercise 9 **16**
9.1 Requirement . . . 16
9.2 Source Code . . . 16

10 Exercise 10 **16**
10.1 Requirement 16
10.2 Integrated Solution 16

1 Exercise 1

1.1 Requirement

Design an FSM for the traffic-light system. The application has four operating modes selected by the first button. In Normal mode, two roads follow a traffic-light sequence. In Modes 2–4, the user can modify the red, amber, and green durations of the main road, respectively.

1.2 FSM Design

The solution separates the mode controller from the traffic-light controller. The main road is denoted by X and the side road by Y.

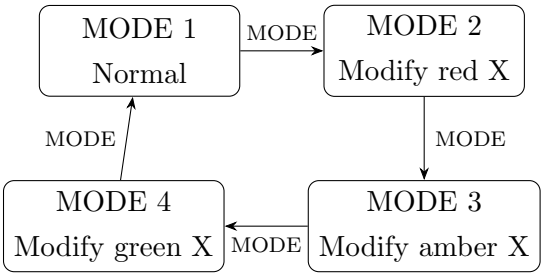


Figure 1.1: Mode FSM controlled by Button 1.

In Mode 1, the traffic controller follows four states. The initial durations are 3 s for green, 2 s for amber, and 5 s for red on both roads.

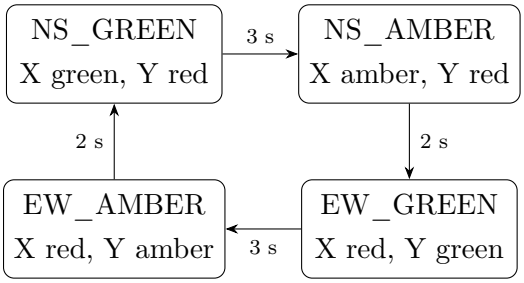


Figure 1.2: Normal traffic FSM with default durations.

The main-road durations are editable. The side-road amber duration is fixed at 2 s, while the other side-road durations are derived as follows:

$$G_Y = R_X - 2, \qquad R_Y = G_X + A_X. \tag{1.1}$$

Leaving Mode 1 pauses the current traffic state and countdown. The saved values are applied at the beginning of the next complete cycle after returning to Mode 1.

2 Exercise 2

2.1 Requirement

Draw a Proteus schematic using 12 traffic LEDs, four seven-segment displays, three buttons, and an STM32 microcontroller.

2.2 Circuit Schematic

The circuit uses an STM32F103C6. Each pair of opposite-facing traffic LEDs shares one output pin. The seven-segment displays are common-anode and are multiplexed using four digit-enable signals.

Table 2.1: GPIO assignments for the traffic-light application.

Function	**GPIO pins**	**Proteus labels**
Traffic lights X	PA2–PA4	LRx, LYx, LGx
Traffic lights Y	PA5–PA7	LRy, LYy, LGy
Seven-segment data	PB0–PB6	SEG0–SEG6
Digit enable	PA12–PA15	EN0–EN3
MODE, INCREASE, SET	PB13–PB15	BUTTON1–BUTTON3

The traffic LEDs are active-low in the tested Proteus schematic. The common-anode segments and digit-enable signals are also active-low. The three buttons are connected as active-low inputs with pull-up resistors.

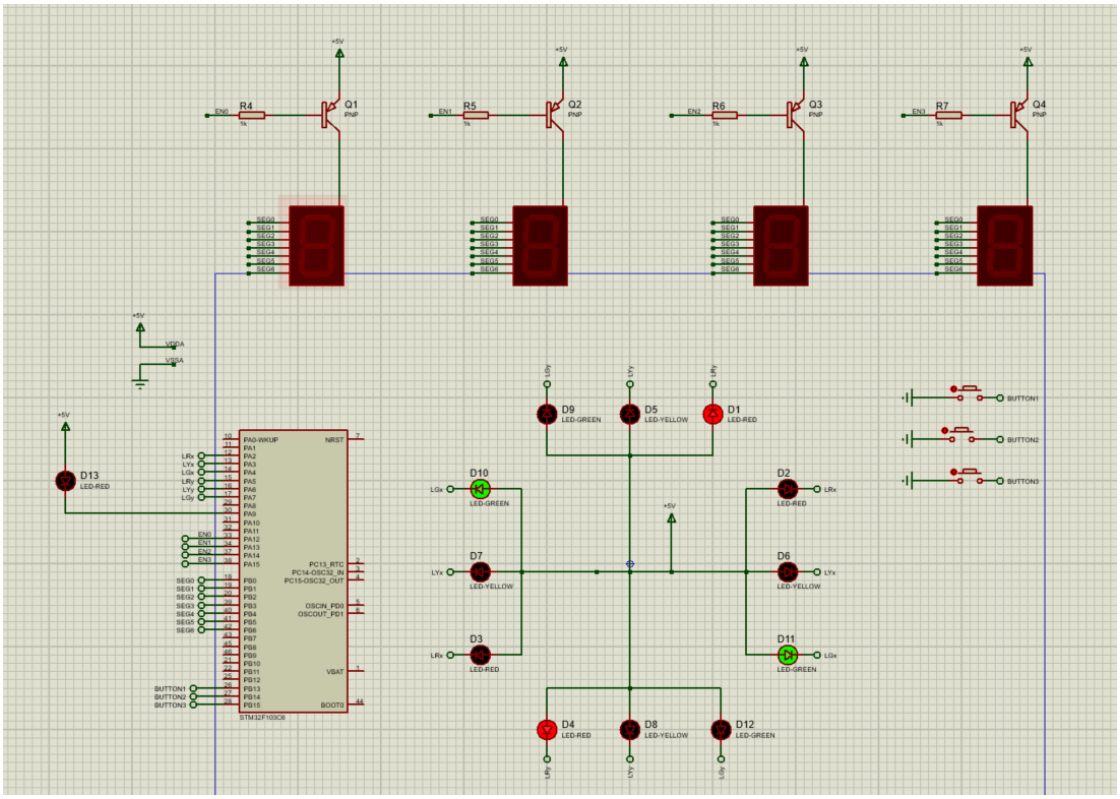


Figure 2.1: Complete Proteus traffic-light schematic.

3 Exercise 3

3.1 Requirement

Create an STM32 project matching the Proteus schematic. Configure GPIO inputs and outputs and generate a timer interrupt approximately every 10 ms.

3.2 STM32 Configuration

The STM32F103C6 uses the internal HSI clock at 8 MHz. TIM2 is configured for button scanning and TIM3 for traffic timing and display multiplexing. TIM2 and TIM3 global interrupts are enabled.

Table 3.1: Hardware timer settings at an 8 MHz timer clock.

Timer	Prescaler	Period	Interval	Purpose
TIM2	7999	9	10 ms	Buttons
TIM3	79	99	1 ms	Display/traffic

The buttons on PB13–PB15 are configured as GPIO inputs with pull-up. The traffic, segment, and digit-enable pins are push-pull GPIO outputs. Debug pins are released for the digit and segment signals in this Proteus configuration.

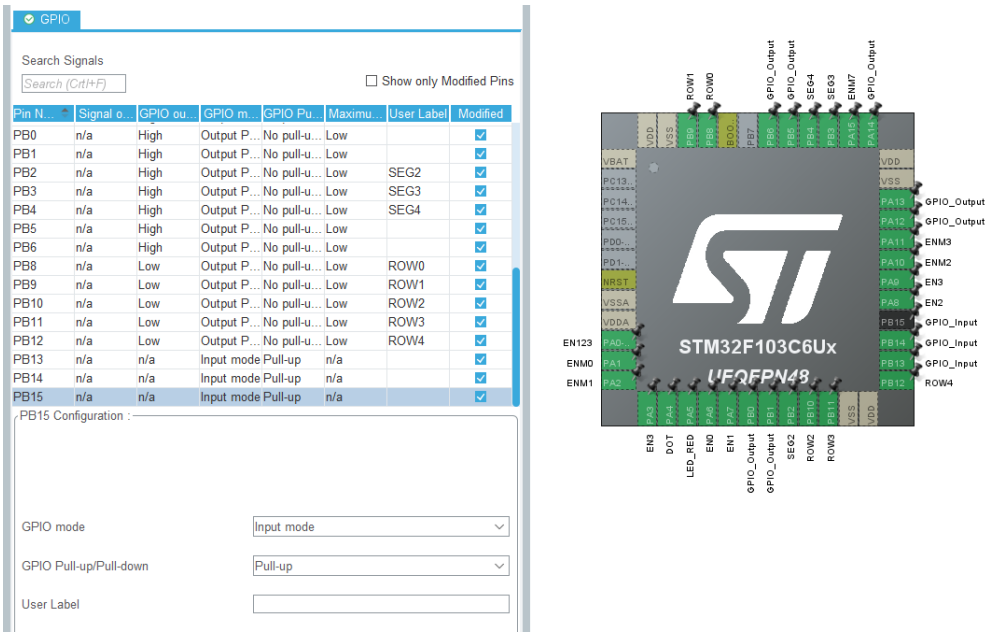


Figure 3.1: STM32CubeMX GPIO pinout for STM32F103C6.

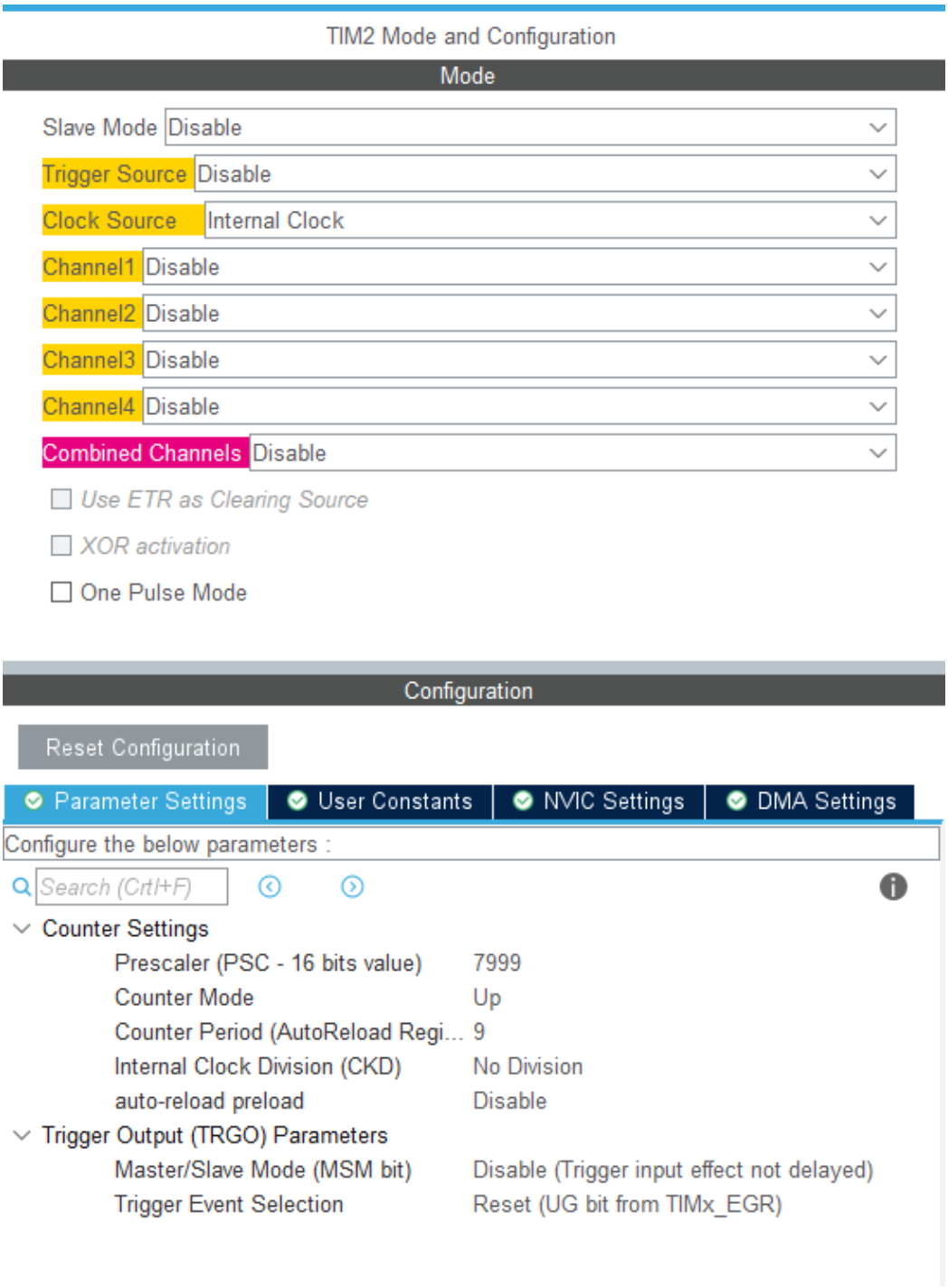


Figure 3.2: TIM2 parameter settings.

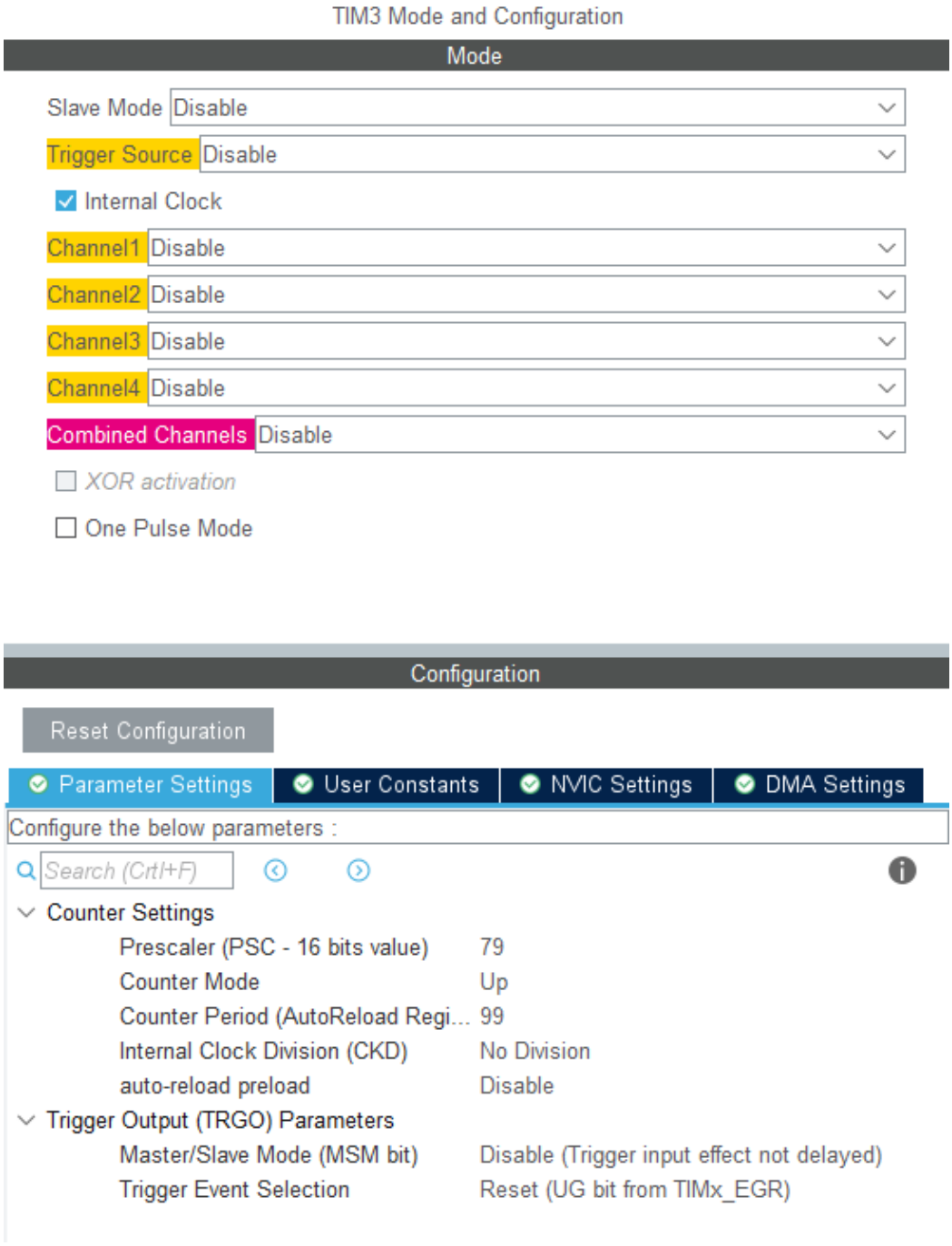


Figure 3.3: TIM3 parameter settings.

3.3 Source Code

The program initializes GPIO and timers before entering the main loop. The interrupt callback separates button input from traffic and display updates.

```
MX_GPIO_Init();
MX_TIM2_Init();
MX_TIM3_Init();
app_init();
HAL_TIM_Base_Start_IT(&htim2);
HAL_TIM_Base_Start_IT(&htim3);

while (1)
{
    fsm_for_input_processing();
}
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2) {
        button_reading();
    } else if (htim->Instance == TIM3) {
        app_tick_1ms();
    }
}
```

4 Exercise 4

4.1 Requirement

Modify timer parameters so that changing the button interrupt interval from 10 ms to 1 ms or 100 ms does not change the 2 Hz LED blinking frequency.

4.2 Source Code

The two functions are handled by separate timers. TIM2 reads buttons, while TIM3 provides a fixed 1 ms time base. A 2 Hz blink has a 500 ms period, so the LEDs are on for 250 ms and off for 250 ms.

```
blink_ms++;
if (blink_ms >= 500U) blink_ms = 0;
```

```
3
4 uint16_t lights = 0;
5 if (blink_ms < 250U) {
6     if (mode == MODE_RED)   lights = GPIO_PIN_2 | GPIO_PIN_5;
7     if (mode == MODE_AMBER) lights = GPIO_PIN_3 | GPIO_PIN_6;
8     if (mode == MODE_GREEN) lights = GPIO_PIN_4 | GPIO_PIN_7;
9 }
10 display_set_lights(lights);
```

For TIM2 with an 8 MHz input clock and prescaler 7999, ARR values 0, 9, and 99 correspond to 1 ms, 10 ms, and 100 ms. Since blinking uses TIM3, changing these TIM2 parameters does not change its period. A longer button-scanning interval may still reduce button responsiveness.

5 Exercise 5

5.1 Requirement

Add modules for input reading and output display, implement button debouncing, and use Button 1 to increase the current operating mode.

5.2 Source Code

The project adds separate modules for input reading, input processing, and output display. TIM2 reads three active-low buttons every 10 ms. A new state is accepted when two consecutive samples match. A press event is generated only when the stable state changes to pressed.

```
1 void button_reading(void)
2 {
3     for (uint8_t i = 0; i < NUMBER_OF_BUTTONS; i++) {
4         GPIO_PinState raw = HAL_GPIO_ReadPin(GPIOB,
              button_pins[i]);
5
6         if (raw == last_raw[i] && raw != stable[i]) {
7             stable[i] = raw;
8
```

```
            if (raw == GPIO_PIN_RESET && press_count[i] < 255
                U) {
                if (i != BUTTON_INCREASE ||
                    !ignore_increase_until_release)
                    press_count[i]++;
            }
        }
        last_raw[i] = raw;
    }
    // Auto-repeat for INCREASE is handled below this block.
}
```

The mode FSM processes the event once, so holding Button 1 does not repeatedly change modes.

```
if (button_take_press(BUTTON_MODE)) {
    button_cancel_edit_events();
    mode = (mode == MODE_GREEN) ?
           MODE_NORMAL : (ModeState)(mode + 1);
    blink_ms = 0;
    error_ms = 0;
    // Load the selected editable duration.
}
```

6 Exercise 6

6.1 Requirement

Display the selected mode using seven-segment LEDs and blink the corresponding traffic LEDs at 2 Hz.

6.2 Source Code

In Modes 2–4, the first two digits show the editable duration and the last two digits show the mode number. All LEDs of the selected color blink together. The digit data are refreshed by TIM3 every 1 ms.

```
if (error_ms > 0U) {
    error_ms--;
    display_set_error((uint8_t)mode);
} else {
    display_set_numbers(edit_value, (uint8_t)mode);
}
display_refresh();
```

For common-anode displays, digit enables and segments are active-low. The display routine disables all digits, updates the segment pattern, and enables one digit for the current scan.

```
HAL_GPIO_WritePin(GPIOA, all_digits, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOB, all_segments, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOB, (uint16_t)(code & 0x7FU),
   GPIO_PIN_RESET);
HAL_GPIO_WritePin(GPIOA, digit_pins[current_digit],
   GPIO_PIN_RESET);
current_digit = (current_digit + 1U) % 4U;
```

7 Exercise 7

7.1 Requirement

Use Button 2 to increase the red-light duration of the main road and Button 3 to save the selected value.

7.2 Source Code

At startup, the main-road red duration is 5 s. In Mode 2, `edit_value` is initialized from `pending.red`. Button 2 increases this value and wraps from 99 back to 1.

```
if (button_take_press(BUTTON_INCREASE) && mode != MODE_NORMAL
   ) {
    edit_value = (edit_value >= 99U) ? 1U : edit_value + 1U;
    error_ms = 0;
}
```

When Button 3 is pressed in Mode 2, the program tests the candidate configuration. A valid value is saved in `pending` and applied at the next complete traffic cycle. The side-road green duration is calculated from the main-road red value.

```
TrafficTime next = pending;
if (mode == MODE_RED) next.red = edit_value;

uint8_t valid = (next.red > SIDE_AMBER_TIME) &&
    ((uint16_t)next.green + next.amber <= 99U);

if (valid) pending = next;
else error_ms = 1000U;
```

For example, increasing red X from 5 s to 6 s changes green Y from 3 s to 4 s after the new configuration becomes active.

8 Exercise 8

8.1 Requirement

Use Button 2 to increase the amber-light duration of the main road and Button 3 to save the selected value.

8.2 Source Code

The same editing FSM is reused. Mode 3 loads the saved amber value from `pending.amber` and updates it when SET is pressed. The side-road amber duration remains fixed at 2 s.

```
if (mode == MODE_AMBER)
    edit_value = pending.amber;

// Inside SET processing:
TrafficTime next = pending;
if (mode == MODE_AMBER)
    next.amber = edit_value;
```

Changing amber X from 2 s to 3 s also increases red Y from 5 s to 6 s, because $R_Y = G_X + A_X$. The candidate must still pass the configuration validation before it is stored.

9 Exercise 9

9.1 Requirement

Use Button 2 to increase the green-light duration of the main road and Button 3 to save the selected value.

9.2 Source Code

Mode 4 uses the same INCREASE and SET logic, with the selected duration stored in `pending.green`. A new value is rejected if the sum of the main-road green and amber durations exceeds 99 s.

```
if (mode == MODE_GREEN)
    edit_value = pending.green;

// Inside SET processing:
TrafficTime next = pending;
if (mode == MODE_GREEN)
    next.green = edit_value;
```

For example, increasing green X from 3 s to 4 s changes red Y from 5 s to 6 s. Invalid SET attempts keep the previous saved values and display `Er` briefly on the first two digits.

10 Exercise 10

10.1 Requirement

Integrate all previous exercises into one STM32 project. Demonstrate normal traffic operation, mode switching, button debouncing, duration editing, and seven-segment output in Proteus. Prepare a video and submit the report and source code.

10.2 Integrated Solution

The final application uses TIM2 for debouncing and TIM3 for traffic timing, blinking, and display multiplexing. The main loop processes button events through the mode FSM.

The current traffic phase and remaining time are preserved when the system leaves Mode 1. The `active` structure stores the timings for the current cycle, while `pending`

stores validated edits. The new configuration is applied only when the normal FSM finishes `EW_AMBER` and returns to `NS_GREEN`.

```
if (phase_ms >= (uint32_t)phase_seconds(phase) * 1000U) {
    phase_ms = 0;
    if (phase == EW_AMBER) {
        phase = NS_GREEN;
        active = pending;
    } else {
        phase = (TrafficState)(phase + 1);
    }
}
```

Table 10.1: Expected behavior of the integrated application.

Operation	**Expected result**
Normal mode	X green 3 s, X amber 2 s, Y green 3 s, Y amber 2 s in the default cycle
MODE button	Cycles through Modes 1, 2, 3, 4, then 1
Editing modes	Selected color blinks at 2 Hz; editable duration and mode are displayed
INCREASE button	Increases the duration; 99 wraps to 1
SET button	Saves a valid value; invalid values show `Er` without changing stored settings
Return to Mode 1	Resumes the paused traffic phase and remaining time
Pending update	Takes effect at the next full cycle

Source Files

https://github.com/Kaytii210/Microprocessors-Microcontrollers/tree/main/Lab03

Video

https://github.com/Kaytii210/Microprocessors-Microcontrollers/blob/main/

Lab03/img/Lab03.mp4